



Software Relevant Tools, Standards, and Code Versioning using Github

Name:	Mones, Quert Benedict V.	Date:	August 22, 2026
Program:	CPE	Section:	B4

A. Learning Outcomes:

At the end of this laboratory, students should be able to:

- 1. Identify software development tools and standards used in software projects.
- 2. Create and manage a GitHub repository.
- 3. Apply Git version control commands.
- 4. Collaborate using GitHub features.
- 5. Demonstrate proper documentation and coding standards.

B. Laboratory Scenario:

You have been hired as a junior software developer in a software company. Your team uses GitHub for source code management and follows coding standards to maintain software quality. Your task is to create a project repository, manage versions of source code, and document your work according to industry standards.

C. Practical Tasks:

1. Software Development Tools and Standards

Create a document named Tools_and_Standards.md containing:

- a. List at least 5 development tools.
- b. Describe the purpose of each tool.
- c. Identify at least three coding standards or best practices used in software development.

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

(Insert images here from the output)

```
1 Software Development Tools and Standards
2
3 Development Tools
4 * Visual Studio Code (VS Code): A lightweight and versatile source code editor widely used for writing code in various languages with built-in debugging and terminal support.
5
6 * Git: A distributed version control system used to track code changes, collaborate on group projects, and manage different versions of code.
7
8 * GitHub: A web-based platform built on top of Git, used by students to host code repositories, submit collaborative projects, and back up work.
9
10 * Python (IDLE / Jupyter Notebook): A core programming environment and interactive notebook tool heavily used in introductory and advanced university courses for data analysis, scripting, and assignments.
11
12 * Dev-C++ / Code::Blocks: Popular lightweight integrated development environments (IDEs) frequently used in college computer science courses for learning and running C and C++ programs.
13
14 Coding Standards and Best Practices
15 * Consistent Naming Conventions: Using clear, descriptive identifiers for variables, functions, and classes to improve code readability.
16 * Modular Code Structure: Breaking down large programs into smaller, manageable, and reusable components to maintain a clean separation of concerns.
17 * Comprehensive Documentation: Writing clear comments and maintaining up-to-date documentation to explain code logic and usage.
```

- 2. GitHub Repository Creation
 - a. Create a GitHub account (if none exists).
 - b. Create a public repository named: Software-Versioning-Lab
 - c. Add the following:
 - i. README.md
 - ii. LICENSE
 - iii. .gitignore
 - d. Upload this laboratory file in this repository once accomplished.
 - e. Submit GitHub repository link
 - f. Screenshot of repository homepage



Laboratory Report 1

Software Relevant Tools, Standards, and Code Versioning using Github

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

(Insert images here from the output)

[quertvmones-ux/Software-Versioning-Lab](#)

3. Git Version Control
- a. Perform the following Git commands and document them in a file named: (Git_Report.md)

i. Clone Repository

a) git clone <repository-url>

ii. Creating New Branch

a) git checkout -b feature-update



Software Relevant Tools, Standards, and Code Versioning using Github

- iii. Modify README.md
 - a) Add your name and laboratory information
- iv. Commit Changes
 - a) git add .
 - b) git commit -m "Updated README with laboratory information"
- v. Push Changes
 - a) git push origin feature-update
- vi. Create Pull Request
 - a) feature-update → main
- b. Screenshot the following:
 - i. Commands executed
 - ii. Pull Request
 - iii. Commit history

(Please include the following: 1. Screenshots of the output)

Screenshot of Output

1 git clone https://github.com/quertvmones-ux/Software-Versioning-Lab

2 cd Software-Versioning-Lab

3 git checkout -b feature-update

4 git add .

5 git commit -m "Updated README with laboratory information"

6 git push origin feature-update

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab> git clone https://github.com/quertvmones-ux/Software-Versioning-Lab

Cloning into 'Software-Versioning-Lab'...

remote: Enumerating objects: 5, done.

remote: Counting objects: 100% (5/5), done.

remote: Compressing objects: 100% (4/4), done.

remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0 (from 0)

Receiving objects: 100% (5/5), done.

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab> ^C

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab> cd Software-Versioning-Lab

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git checkout -b feature-update

Switched to a new branch 'feature-update'

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git add .

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git commit -m "Updated README with laboratory information"

On branch feature-update

nothing to commit, working tree clean

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git push origin feature-update

Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)

remote:

remote: Create a pull request for 'feature-update' on GitHub by visiting:

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git checkout -b feature-update

Switched to a new branch 'feature-update'

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git add .

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git commit -m "Updated README with laboratory information"

On branch feature-update

nothing to commit, working tree clean

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git push origin feature-update

Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)

remote:

remote: Create a pull request for 'feature-update' on GitHub by visiting:

remote: https://github.com/quertvmones-ux/Software-Versioning-Lab/pull/new/feature-update

remote:

To https://github.com/quertvmones-ux/Software-Versioning-Lab

* [new branch] feature-update -> feature-update

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git checkout feature-update

Already on 'feature-update'

PS C:\Users\User\OneDrive\Documents\GitHub\Software-Versioning-Lab\Software-Versioning-Lab> git status

>> git log -n 2 --oneline

On branch feature-update



Laboratory Report 1

Software Relevant Tools, Standards, and Code Versioning using Github

quertvmones-ux / Software-Versioning-Lab

CodeIssuesPull requestsActionsProjectsWikiSecurity and qualityInsightsSettings

Search

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

basic: main

compare: feature-update

Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#)

Create pull request

2 commits

2 files changed

At 1 contributor

Commits on Aug 22, 2026

Create .gitignore

quertvmones-ux authored 4 minutes ago

Verified656248c

Update README.md

quertvmones-ux authored now

Verified656243b

Showing 2 changed files with 3 additions and 0 deletions.

1 .gitignore

+++ -- -6,0 +1,0

1 +

2 README.md

+++ -- -1,2 +1,4

1 # Software-Versioning-Lab

2 + Quert Benedict V. Nones

3 + Laboratory report 1: Software Relevant Tools, standards, and code versioning using github

2 4

© 2026 GitHub, Inc.

TermsPrivacySecurityStatusCommunityDocsContactManage cookiesDo not share my personal information

quertvmones-ux / Software-Versioning-Lab

CodeIssuesPull requests1ActionsProjectsWikiSecurity and qualityInsightsSettings

Search

Commits

feature-update

All usersAll time

Commits on Aug 22, 2026

Update README.md

quertvmones-ux authored 2 minutes ago

Verified656243b

Create .gitignore

quertvmones-ux authored 7 minutes ago

Verified956248c

Initial commit

quertvmones-ux committed 1 hour ago

9596e99

CPE106L-4 Software Design Laboratory

School of Artificial Intelligence, Electrical, Electronics, and Computer Engineering

Page | 4

Software Relevant Tools, Standards, and Code Versioning using Github

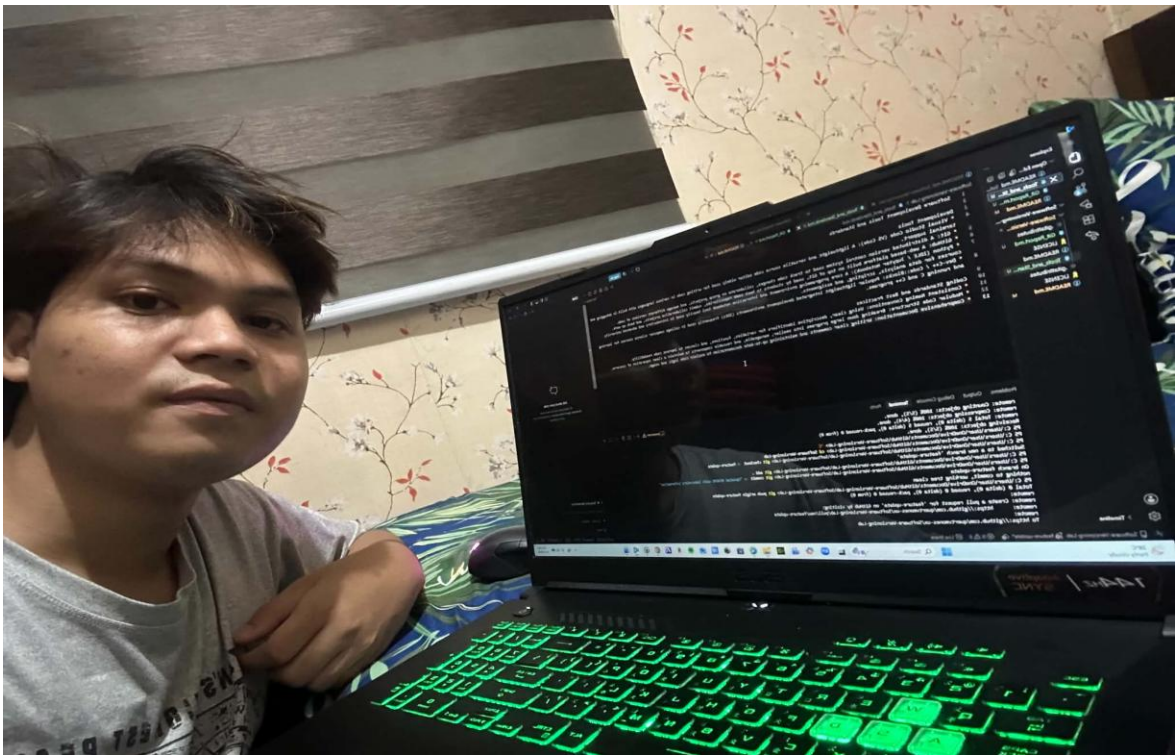
D. Findings, Observations, and Comments:

Guide Questions:

- 1. What additional steps are taken in this activity?
- 2. Are there any difficulties during the GitHub setup?
- 3. What are your findings, observations, and comments in this exercise?

Throughout this activity, I walked through the complete version control workflow, which involved initializing a remote GitHub repository, cloning it to my local machine, and setting up a dedicated workspace. Specifically, I created and switched to a feature-update branch, modified the project's documentation file to include my student details, staged those changes, committed them with a descriptive message, and pushed the updated branch back to the remote server. I did run into a few hurdles along the way, particularly when dealing with synchronization issues and getting GitHub's interface to recognize my branch updates for the pull request. Navigating those roadblocks required careful troubleshooting, such as verifying my branch status, managing my commits, and making sure the file modifications were correctly registered before opening the pull request to merge everything back into the main branch. Overall, this exercise gave me a much deeper, hands-on appreciation for how Git and GitHub operate in real-world scenarios. It showed me firsthand how essential version control is for tracking project history, avoiding code conflicts, and safely managing collaborative updates step by step.

PICTURE/PROOF OF DOING THE ACTIVITY





Software Relevant Tools, Standards, and Code Versioning using Github

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indentation, but inappropriate elements and structures and poor program documentation.	The program had corrected indentation, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indentation, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings, observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings, observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	